Conversation Log: Construction Project Management Timesheet Websites

Date: October 30, 2026

User Prompt 1

I am a civil engineering major creating a beginner Python website with the Drafter library. I want to make a construction site timesheet tracker. A user should enter a worker’s name, trade, hours worked per day, and number of days worked. What useful features should a simple version have, what calculations should it perform, and what existing tools are used to track construction worker time? Please keep the suggestions realistic for an introductory programming project and do not provide code.

Assistant Response 1

Building a construction site timesheet tracker as an introductory Python project using the Drafter library is a fantastic way to bridge civil engineering concepts with software development. In construction, labor tracking is one of the most vital components of project management since labor often accounts for a major portion of a project's budget.

Realistic and helpful features, calculations, and industry tools for a beginner-friendly project include the following:

1. Useful Features for a Simple Version

Given the inputs you specified (worker's name, trade, hours per day, and number of days worked), a beginner web app can incorporate these practical features:

- **Worker Directory / Summary View:** A clean table or list that displays all submitted worker entries so a site superintendent or project manager can see everyone on-site at a glance.
- **Trade Filtering or Categorization:** The ability to sort or filter workers by their trade (e.g., Carpenter, Electrician, Laborer, Operator) to see how many labor resources are allocated to specific crafts.
- **Input Validation & Error Handling:** Simple checks to prevent a user from submitting negative numbers for hours or days, or leaving text fields blank.
- **Reset / Clear Functionality:** A way to clear individual entries or reset the active timesheet session.
- **Persistent Session or Summary Totals:** Displaying aggregate metrics at the top of the page, such as total workforce headcount and grand total hours logged for the entire site.

2. Essential Calculations to Perform

Beyond simply multiplying hours by days, a construction timesheet tracker should perform calculations that reflect real-world civil and construction engineering workflows:

- **Total Individual Hours:** Multiplying the *hours worked per day* by the *number of days worked* to find the total gross hours for an individual worker.
- **Estimated Gross Pay (by Trade/Hourly Rate):** Associating a standard base hourly wage depending on the trade (e.g., a master electrician typically commands a different hourly rate than a general laborer) and calculating total estimated labor cost per worker.
- **Overtime Estimation:** A standard work week in construction is often capped at 40 hours before overtime kicks in (typically paid at 1.5x, known as "time-and-a-half"). Your app can evaluate if a worker's total hours exceed a standard threshold and separate regular hours from overtime hours.
- **Aggregated Labor Cost:** Summing up the total labor expenditures across all workers to show project managers what the current payroll liability is for that tracking period.

3. Existing Tools Used in the Industry

In professional practice, construction companies have moved far beyond paper cards, though the underlying logic remains the same. Real-world tools include:

- **Specialized Construction Time-Tracking Software:** Tools like **ClockShark**, **Raken**, and **ExakTime (Arcoro)** are built specifically for the field. They often include mobile app clock-ins, GPS tracking, and geofencing to ensure workers are physically on the job site.
- **Enterprise Heavy Construction Software:** Platforms like **HCSS (HeavyJob)** or **Contractor Foreman** are widely used by civil engineering and heavy highway contractors. They link time cards directly to specific "cost codes" (standardized numerical codes used in estimating and bidding, like earthwork vs. concrete pouring).
- **General Spreadsheets (Excel / Google Sheets):** Despite dedicated software, many small-to-midsize subcontractors still rely heavily on customized Excel templates because they offer flexible calculation layouts similar to what you are building in Python.

Focusing your project on clear data organization, distinct trade wage logic, and overtime thresholds will give you a solid foundation that mirrors how real construction management software evaluates field productivity.

User Prompt 2

GPS tracking, automatic clock-in and clock-out, payroll calculations, overtime rules, and employee scheduling are too complicated for my introductory Python project. Can you narrow the idea to a basic timesheet where I manually enter each worker's name, trade, hours per day, and days worked? I only need the program to calculate total hours and show summaries by worker and trade. The idea is to manually enter timesheet information and use basic arithmetic, lists, loops, and conditionals to organize it. The website should not connect to payroll systems, track worker locations, or determine overtime.

Assistant Response 2

Narrowing your project scope to a clean, foundational Minimum Viable Product (MVP) is a smart approach for an introductory programming class. Focusing strictly on manual entry and core arithmetic allows you to master the fundamentals of Python and the Drafter library without getting bogged down by complex business logic.

A streamlined, beginner-friendly specification for your manual timesheet tracker includes the following components:

1. Core Features for a Basic Manual Tracker

With a focus on manual entry and straightforward data organization, your Drafter web application can feature:

- **Simple Data Entry Form:** A form with fields for worker name, trade (e.g., text input or a dropdown), hours worked per day, and number of days worked.
- **Master Timesheet Log:** A running table that displays every entry submitted during the active session, showing each worker's individual details right as they are added.
- **Worker Summary View:** A clean breakdown showing each individual worker alongside their cumulative total hours.
- **Trade Summary View:** A grouped breakdown showing each trade category (e.g., Laborer, Carpenter, Operator) and the grand total of hours logged across all workers in that trade.

2. Simple Calculations to Perform

Without payroll integrations or overtime rules, your math remains strictly focused on basic arithmetic:

- **Individual Total Hours:**

- $TotalHours = HoursperDay \times DaysWorked$
-
- **Trade Subtotals:** Summing the individual total hours for all workers sharing the same trade designation.
- **Site Grand Total:** Summing the total hours of every worker on the site to give a single aggregate number for total labor hours expended.

3. How Core Python Concepts Will Power Your App

Your project will naturally map directly to the foundational programming concepts you are learning:

- **Lists and Data Structures:** You can store each submitted timesheet entry (using a dictionary or parallel lists) so that the app remembers multiple workers across form submissions.
- **Loops (for / while):** You will use loops to iterate through your list of worker entries to calculate summary totals and dynamically render the rows in your summary tables.
- **Conditionals (if / else):** Conditionals will help you categorize workers by trade (e.g., checking if a worker's trade matches "Carpenter" to group their hours together) or validating that text fields aren't accidentally left blank.

This refined scope gives you a robust, fully functional civil engineering application that highlights essential programming logic while remaining entirely manageable for an introductory project.

Separate Conversation

1. User Query

"What are some websites that help construction project managers calculate time sheets"

2. Assistant Response

Construction project managers rely on dedicated digital time-tracking tools to replace manual paper sheets, auto-calculate hours and overtime, allocate costs to specific job codes, and sync directly with payroll software.

Specialized Construction Time Tracking Tools

Website / Tool	Best Used For	Key Features
ClockShark	Small to mid-size subcontractors & trade crews	GPS geofencing, drag-and-drop worker scheduling, cost-code tracking, and direct integration with QuickBooks and ADP.
busybusy	Heavy civil, equipment operators, and sub-crews	Tracks machine/equipment utilization alongside labor hours, operates offline for remote job sites, and handles safety sign-offs.
WorkMax	Mid-to-large general contractors & enterprise teams	Built for complex multi-state prevailing wage calculations, certified payroll, high-volume cost coding, and enterprise ERP integration.
ExakTime	Field crews needing rugged options	Field-ready mobile app, physical rugged time clocks (Jobclock), automated overtime alerts, and photo ID clock-in verification to stop "buddy punching."
SmartBarrel	High-occupancy jobsites & hardware-first logging	Provides solar-powered, weather-resistant physical kiosk boxes for jobsites with facial recognition clock-in; no worker app required.

All-in-One Construction Management Platforms

If you are already managing project schedules, RFIs, and submittals, these platforms include built-in field timesheet calculation features:

- **Raken:** Combines daily reporting with field time tracking. Foremen can log crew hours, assign cost codes, and submit daily logs straight from their phones.
- **Buildertrend:** Popular among residential builders and remodelers; includes an integrated mobile clock-in/clock-out feature linked directly to project estimates and budgets.
- **Procore:** Enterprise construction management platform with a dedicated **Timecard** module that links labor cost data directly to real-time project budget tracking.

Free & Budget-Friendly Options

- **Jibble:** Free tier includes GPS tracking, facial recognition kiosk mode, automated break tracking, and overtime calculation.
- **Clockify:** Offers a comprehensive free plan with custom project tagging, time tracking, and exportable timesheets for small teams.

Complete Conversation & Project Documentation

Developer: Chasiti Bobet-Diaz (chasbd@udel.edu)

Project: Construction Site Timesheet Tracker

Course: CS1 Final Project

Part 1: Transcript of Conversation

User Prompt 1

Based on this project, fill out these questions...

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	Implementation
Purpose/Functionality	A 1-2 sentence description of the purpose or functionality of your site
State Dataclass Attributes	Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.
Second Dataclass	Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)

Routes (minimum 7)	The names of your route functions
Pages (minimum 5)	A brief description of the pages in your site
User Input (minimum 3)	Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes
If Statements (minimum 3)	Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site
Loop (minimum 1)	This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site

[Attached document: CBD - CS1 Final Project - Pick a Problem (1).docx]

Assistant Response 1

Here are the completed answers for your Construction Site Timesheet Tracker project, formatted and ready to be transferred into your project assignment document.

2) Implementing Requirements

Purpose/Functionality

This web application allows construction project managers and superintendents to manually log workforce hours by worker name and trade, automatically calculate individual and aggregate labor hours, and review organized summaries by worker and trade to simplify timesheet record-keeping.

State Dataclass Attributes

- entries (list[TimesheetEntry]): A list holding instances of the TimesheetEntry dataclass. It is modified when users submit new entries or delete incorrect ones.
- site_name (str): Stores the name of the construction project. Modified when the user initializes or updates the site details.

- pay_period (str): Represents the active timesheet period. Modified when setting up the tracking cycle.
- error_message (str): Holds feedback strings. Modified whenever input validation fails or passes successfully.

Second Dataclass (TimesheetEntry)

- **Attributes & Types**:
  - worker_name (str): The name of the worker.
  - trade (str): The construction trade (e.g., Carpenter, Electrician, Laborer, Operator).
  - hours_per_day (float): Average hours worked per day.
  - days_worked (int): Number of days worked in the period.
  - total_hours (float): Calculated total gross hours (hours_per_day * days_worked).
- **Modification**: Created upon valid form submission and appended to the State's entries list; individual entries can be removed via deletion handlers.

Routes (Minimum 7)

1. home_route (/)
2. add_entry_route (/add)
3. submit_entry_route (/submit)
4. view_entries_route (/entries)
5. trade_summary_route (/summary)
6. delete_entry_route (/delete/{index})
7. help_route (/help)

Pages (Minimum 5)

1. **Home Dashboard**: Central hub displaying quick navigation and project overview statistics.
2. **Add Entry Page**: Form interface for inputting worker details, trade selection, and hours.
3. **View Entries Page**: Detailed table showing all recorded workforce entries with management options.
4. **Trade Summary Page**: Analytical breakdown aggregating total hours per construction trade and grand total site hours.
5. **Help & Instructions Page**: Guide explaining how to record data and interpret totals.

User Input (Minimum 3)

1. TextBox for Worker Name: Captures the individual employee's name for record

identification.
2. SelectBox for Trade Selection: Restricts input to predefined construction trades (Carpenter, Electrician, Laborer, Operator) to maintain consistent data categorization.
3. TextBox for Hours Worked per Day: Accepts numerical input representing daily shift hours.
4. TextBox for Days Worked: Accepts numerical input representing total days worked during the period.

If Statements (Minimum 3)

1. **Input Validation Check**: Evaluates if the worker name is blank or numerical values are negative/zero; if true, it triggers an error message and prevents entry creation.
2. **Trade Filtering / Subtotal Logic**: Evaluates whether a specific entry's trade matches a target category during summary aggregation to compute accurate trade subtotals.
3. **List Empty Check**: Evaluates if the entries list is empty before rendering summary tables, displaying a friendly placeholder message if no data exists yet.

Loop (Minimum 1)

- **Timesheet Aggregation Loop**: Iterates over the entries list in the State dataclass to calculate the grand total site hours and individual worker total hours (hours_per_day * days_worked), ensuring all submitted records are processed dynamically.

Drafter Timesheet Tracker — Conversation Documentation

Purpose: Record the troubleshooting conversation about launching the construction timesheet tracker in Thonny.

Note: The full application source code was supplied in the conversation. A copy of the latest pasted source is included at the end of this document when available.

User
Based on the provided attachments and project requirements, draft code for the construction timesheet tracker.

Assistant
Provided a proposed Drafter application with a State dataclass, TimesheetEntry dataclass, helper functions, routes for project setup, worker entries, viewing and deleting entries, trade summaries, and help. The app was started with start_server(State(...)).

User
The code is running in Drafter as "Hello world! Welcome to Drafter."

Assistant
Explained that Drafter appeared to be running its default page rather than the timesheet app. Suggested checking that the correct file was open and that the start_server(...) call was at the bottom of that file.

User
Uploaded the complete timesheet application source code.

Assistant
Reviewed the code and suggested stopping the old server, saving the correct file, running it directly in Thonny, and refreshing the local website.

User
thonny

Assistant
Gave Thonny-specific steps: open the correct file, save it, stop the old server, press F5, and open the local URL shown in the Shell.

User
Shared the Shell output: Drafter ready at http://localhost:8000; GET / returned 200 OK; WebSocket connection accepted; GET /main.py returned 304 Not Modified.

Assistant
Noted that the server had started successfully and asked the user to check the correct route and browser page.

User
Reported that the page still displayed Hello World and shared additional Shell output, including POST /__drafter_error_log.

Assistant
Identified that the main route should be named index for the root page. Recommended renaming home_route to index and updating buttons that referred to home_route.

User
The program works now, but Thonny warned: Line 365: Redefining name 'index' from outer scope (line 167); Line 408: Redefining name 'index' from outer scope (line 167).

Assistant
Explained that these were warnings rather than runtime errors. Recommended keeping index as the home route name and renaming local loop variables and function parameters to a clearer name such as entry_number. Also explained that the Argument name in the Delete button must match the renamed function parameter.

User
Create a copy of this conversation for documentation.

Troubleshooting outcome
The application began displaying the intended timesheet tracker after the home page route was renamed from home_route to index. This makes the route available at the root address, http://localhost:8000/.

Thonny's later redefined-outer-name warnings were caused by using the name index both for the home route and for local variables or parameters. The suggested cleanup is to rename those local names to entry_number and update all matching references, including the form/button Argument name used by delete_entry.

Relevant Thonny Shell output
```
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Drafter ready at http://localhost:8000
INFO:     ::1:55115 - "GET / HTTP/1.1" 200 OK
INFO:     ::1:55115 - "GET / HTTP/1.1" 200 OK
INFO:     ::1:55117 - "WebSocket /__drafter_ws" [accepted]
INFO:     connection open
INFO:     ::1:55115 - "GET /main.py HTTP/1.1" 304 Not Modified
INFO:     ::1:55118 - "WebSocket /__drafter_ws" [accepted]
```

```
INFO:     connection open
INFO:     ::1:55115 - "POST /__drafter_error_log HTTP/1.1" 200 OK
```

Appendix: Latest pasted application source

The source below is reproduced from the latest code attachment shared during troubleshooting.

```
from drafter import *
from dataclasses import dataclass
from bakery import assert_equal
# ----------------------------------------
# WEBSITE INFORMATION
# ----------------------------------------
set_site_information(
   author="chasbd@udel.edu",
   description="A construction site timesheet tracker that "
               "records worker hours and summarizes labor.",
   sources="Drafter documentation and CS1 course materials.",
   planning= "CBD - CS1 Final Project - Design.docx" "CBD - CS1 Final Project - Pick a Problem.docx",
   links=["https://github.com/chasbd-ud/construction-timesheet-tracker"]
)
hide_debug_information()
set_website_title("Construction Site Timesheet Tracker")
set_website_framed(False)
# ----------------------------------------
# CONSTANTS
# ----------------------------------------
TRADES = [
   "Carpenter",
   "Electrician",
   "Laborer",
   "Operator"
]
# ----------------------------------------
# DATA CLASSES
# ----------------------------------------
@dataclass
class TimesheetEntry:
   """
   Stores the recorded hours for one construction worker.
   """
   worker_name: str
   trade: str
   hours_per_day: float
   days_worked: int
   total_hours: float
@dataclass
class State:
   """
   Stores all information for the active construction timesheet.
   Attributes:
       entries: List of worker timesheet entries.
       site_name: Name of the construction project.
       pay_period: Current timesheet period.
       error_message: Feedback displayed to the user.
   """
   entries: list[TimesheetEntry]
   site_name: str
   pay_period: str
   error_message: str
# ----------------------------------------
# HELPER FUNCTIONS
# ----------------------------------------
def calculate_total_hours(
```

```
    hours_per_day: float,
    days_worked: int
) -> float:
    """
    Calculates the total hours worked by one employee.
    """
    return hours_per_day * days_worked
def calculate_site_hours(
    entries: list[TimesheetEntry]
) -> float:
    """
    Calculates the combined hours for the entire construction site.
    """
    total = 0.0
    for entry in entries:
        total += entry.total_hours
    return total
def calculate_trade_hours(
    entries: list[TimesheetEntry],
    selected_trade: str
) -> float:
    """
    Calculates the total hours recorded for one construction trade.
    """
    total = 0.0
    for entry in entries:
        if entry.trade == selected_trade:
            total += entry.total_hours
    return total
def is_valid_hours(value: str) -> bool:
    """
    Checks whether hours are a positive number no greater than 24.
    Allows whole numbers and decimal values.
    """
    if not value:
        return False
    if value.count(".") > 1:
        return False
    parts = value.split(".")
    for part in parts:
        if part and not part.isdigit():
            return False
    if not parts[0]:
        return False
    hours = float(value)
    if hours <= 0 or hours > 24:
        return False
    return True
def make_entry(
    worker_name: str,
    trade: str,
    hours: float,
    days: int
) -> TimesheetEntry:
    """
    Creates a timesheet entry and calculates its total hours.
    """
    total = calculate_total_hours(hours, days)
    return TimesheetEntry(
        worker_name,
        trade,
        hours,
        days,
        total
    )
# ----------------------------------------
```

```
# HOME DASHBOARD
# ----------------------------------------
@route
def home_route(state: State) -> Page:
   """
   Displays the main construction timesheet dashboard.
   """
   total_workers = len(state.entries)
   total_hours = calculate_site_hours(state.entries)
   return Page(
       state,
       [
           Header("Construction Site Timesheet Tracker"),
           "Welcome to your construction workforce dashboard!",
           HorizontalRule(),
           Header("Project Information", 2),
           "Construction Site: " + state.site_name,
           "Timesheet Period: " + state.pay_period,
           Header("Workforce Overview", 2),
           "Worker Entries: " + str(total_workers),
           "Total Labor Hours: " + str(total_hours),
           HorizontalRule(),
           Header("Navigation", 2),
           Button("Project Setup", "setup_route"),
           Button("Add Worker Entry", "add_entry_route"),
           Button("View Timesheets", "view_entries_route"),
           Button("Trade Summary", "trade_summary_route"),
           Button("Help & Instructions", "help_route"),
       ]
   )
# ----------------------------------------
# PROJECT SETUP
# ----------------------------------------
@route
def setup_route(state: State) -> Page:
   """
   Displays the form for entering project information.
   """
   return Page(
       state,
       [
           Header("Project Setup"),
           "Enter the information for your construction project.",
           "Construction Site Name:",
           TextBox("site_name", state.site_name),
           "Timesheet Period:",
           TextBox("pay_period", state.pay_period),
           Button("Save Project Information", "save_setup"),
           Button("Back to Dashboard", "home_route")
       ]
   )
@route
def save_setup(
   state: State,
   site_name: str,
   pay_period: str
) -> Page:
   """
   Saves the construction site and timesheet period.
   """
   if not site_name.strip():
       state.error_message = "Please enter a construction site name."
   elif not pay_period.strip():
       state.error_message = "Please enter a timesheet period."
   else:
       state.site_name = site_name.strip()
```

```
        state.pay_period = pay_period.strip()
        state.error_message = ""
    return Page(
        state,
        [
            Header("Project Setup"),
            state.error_message,
            Button("Return to Dashboard", "home_route"),
            Button("Edit Project Information", "setup_route")
        ]
    )
# ----------------------------------------
# ADD WORKER ENTRY
# ----------------------------------------
@route
def add_entry_route(state: State) -> Page:
    """
    Displays the form for recording a worker's hours.
    """
    return Page(
        state,
        [
            Header("Add Worker Timesheet Entry"),
            "Enter the worker's information below.",
            "Worker Name:",
            TextBox("worker_name", ""),
            "Construction Trade:",
            SelectBox("trade", TRADES, TRADES[0]),
            "Hours Worked Per Day:",
            TextBox("hours_per_day", "8"),
            "Number of Days Worked:",
            TextBox("days_worked", "5"),
            Button("Submit Timesheet", "submit_entry"),
            Button("Back to Dashboard", "home_route")
        ]
    )
@route
def submit_entry(
    state: State,
    worker_name: str,
    trade: str,
    hours_per_day: str,
    days_worked: str
) -> Page:
    """
    Validates worker information and adds a timesheet entry.
    """
    worker_name = worker_name.strip()
    if not worker_name:
        state.error_message = "Please enter the worker's name."
    elif trade not in TRADES:
        state.error_message = "Please select a valid construction trade."
    elif not is_valid_hours(hours_per_day):
        state.error_message = "Hours must be greater than 0 and no more than 24."
    elif not days_worked.isdigit() or int(days_worked) <= 0:
        state.error_message = "Days worked must be a positive whole number."
    else:
        hours = float(hours_per_day)
        days = int(days_worked)
        new_entry = make_entry(
            worker_name,
            trade,
            hours,
            days
        )
        state.entries.append(new_entry)
```

```
        state.error_message = "Timesheet entry successfully added!"
    return Page(
        state,
        [
            Header("Timesheet Submission"),
            state.error_message,
            Button("Add Another Worker", "add_entry_route"),
            Button("View All Entries", "view_entries_route"),
            Button("Return Home", "home_route")
        ]
    )
# ----------------------------------------
# VIEW TIMESHEET ENTRIES
# ----------------------------------------
@route
def view_entries_route(state: State) -> Page:
    """
    Displays all recorded worker timesheet entries.
    """
    content = [
        Header("Recorded Timesheet Entries"),
        "Construction Site: " + state.site_name,
        "Timesheet Period: " + state.pay_period,
        HorizontalRule()
    ]
    if not state.entries:
        content.append("No worker entries have been recorded yet.")
    else:
        table_data = []
        for index, entry in enumerate(state.entries):
            table_data.append([
                entry.worker_name,
                entry.trade,
                str(entry.hours_per_day),
                str(entry.days_worked),
                str(entry.total_hours),
                Button(
                    "Delete",
                    "delete_entry",
                    Argument("index", index)
                )
            ])
        content.append(
            Table([
                [
                    "Worker",
                    "Trade",
                    "Hours/Day",
                    "Days",
                    "Total Hours",
                    "Action"
                ]
            ] + table_data)
        )
        content.append(
            "Total Site Hours: " +
            str(calculate_site_hours(state.entries))
        )
    content.append(Button("Add Worker", "add_entry_route"))
    content.append(Button("Back to Dashboard", "home_route"))
    return Page(state, content)
# ----------------------------------------
# DELETE TIMESHEET ENTRY
# ----------------------------------------
@route
def delete_entry(state: State, index: int) -> Page:
```

```
    """
    Removes a selected timesheet entry from the list.
    """
    if index >= 0 and index < len(state.entries):
        removed_worker = state.entries[index].worker_name
        state.entries.pop(index)
        state.error_message = (
            removed_worker + "'s entry was successfully removed."
        )
    else:
        state.error_message = "That timesheet entry could not be found."
    return Page(
        state,
        [
            Header("Manage Timesheet Entries"),
            state.error_message,
            Button("View Remaining Entries", "view_entries_route"),
            Button("Return Home", "home_route")
        ]
    )
# ----------------------------------------
# TRADE SUMMARY
# ----------------------------------------
@route
def trade_summary_route(state: State) -> Page:
    """
    Calculates and displays labor hour totals for each trade.
    """
    content = [
        Header("Construction Trade Summary"),
        "Labor hours organized by construction trade.",
        HorizontalRule()
    ]
    if not state.entries:
        content.append("There are currently no timesheet entries.")
    else:
        summary_data = []
        for trade in TRADES:
            trade_hours = calculate_trade_hours(
                state.entries,
                trade
            )
            summary_data.append([
                trade,
                str(trade_hours)
            ])
        content.append(
            Table([
                ["Construction Trade", "Total Hours"]
            ] + summary_data)
        )
        content.append(HorizontalRule())
        content.append(
            Header(
                "Total Site Labor Hours: " +
                str(calculate_site_hours(state.entries)),
                2
            )
        )
        content.append(
            "Total Worker Entries: " + str(len(state.entries))
        )
    content.append(Button("View All Workers", "view_entries_route"))
    content.append(Button("Back to Dashboard", "home_route"))
    return Page(state, content)
# ----------------------------------------
```

```
# HELP PAGE
# ----------------------------------------
@route
def help_route(state: State) -> Page:
   """
   Explains how to use the construction timesheet tracker.
   """
   return Page(
       state,
       [
           Header("Help & Instructions"),
           Header("1. Set Up Your Project", 2),
           "Enter your construction site name and timesheet period.",
           Header("2. Add Worker Information", 2),
           "Enter the worker's name, construction trade, "
           "hours worked per day, and number of days worked.",
           Header("3. Review Timesheets", 2),
           "View all recorded worker entries and their "
           "calculated total hours.",
           Header("4. Review Trade Summaries", 2),
           "See the combined labor hours for Carpenters, "
           "Electricians, Laborers, and Operators.",
           Header("5. Correct Mistakes", 2),
           "Use the Delete button to remove an incorrect entry.",
           HorizontalRule(),
           Header("Important Information", 2),
           "This website records manually entered labor hours. "
           "It does not calculate official payroll, taxes, "
           "benefits, or overtime.",
           Button("Return to Dashboard", "home_route")
       ]
   )
# ----------------------------------------
# UNIT TESTS
# ----------------------------------------
# Test the total hours calculation.
assert_equal(calculate_total_hours(8.0, 5), 40.0)
assert_equal(calculate_total_hours(10.0, 4), 40.0)
assert_equal(calculate_total_hours(6.5, 3), 19.5)
# Test the site-wide hours calculation.
test_entries = [
   TimesheetEntry("John", "Carpenter", 8.0, 5, 40.0),
   TimesheetEntry("Sarah", "Electrician", 8.0, 4, 32.0),
   TimesheetEntry("Mike", "Carpenter", 6.0, 5, 30.0)
]
assert_equal(calculate_site_hours(test_entries), 102.0)
# Test the trade summary calculation.
assert_equal(
   calculate_trade_hours(test_entries, "Carpenter"),
   70.0
)
assert_equal(
   calculate_trade_hours(test_entries, "Electrician"),
   32.0
)
assert_equal(
   calculate_trade_hours(test_entries, "Operator"),
   0.0
)
# Test input validation.
assert_equal(is_valid_hours("8"), True)
assert_equal(is_valid_hours("7.5"), True)
assert_equal(is_valid_hours("0"), False)
assert_equal(is_valid_hours("-5"), False)
assert_equal(is_valid_hours("25"), False)
assert_equal(is_valid_hours("abc"), False)
```

```
# ----------------------------------------
# START WEBSITE
# ----------------------------------------
start_server(
   State(
       entries=[],
       site_name="Construction Timesheet Tracker",
       pay_period="Current Pay Period",
       error_message=""
   )
)
```